

SIMATS ENGINEERING
Department of Computer Science and Engineering
Assignment Questions

Course Code & Name: MLA0101 – Artificial Intelligence and Expert System

Problem Statement:

Design and implement “AgriAdvisor,” an AI-based farm decision-support system that helps small-scale farmers manage irrigation scheduling, pest/disease alerts, and crop-disease diagnosis, addressing the problem of delayed, inconsistent manual advice from field agents. For each sub-task, first formulate the problem using the PEAS (Performance measure, Environment, Actuators, Sensors) framework and justify the most apt agent strategy — simple reflex, model-based reflex, goal-based, or utility-based — given the nature of the environment (fully/partially observable, deterministic/stochastic, static/dynamic). Design corresponding software agents with clearly defined percepts, actions, and internal state, and specify their architecture. Build a rule-based expert system with a knowledge base of IF-THEN production rules (covering crop symptoms, environmental readings, and treatment/action recommendations) and an inference engine using forward and/or backward chaining, so that the agents can query the expert system and act on its recommendations to diagnose crop diseases and advise on irrigation and pest control end-to-end.

Assessment Rubrics

Assignment Title:	A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory
Course Outcome(s) – CO:	CO3: Provide the apt agent strategy to solve a given problem. CO4: Design software agents to solve a problem. CO5: Implement a rule-based expert system.
Bloom’s Taxonomy Level	L3 – Apply; L4 – Analyse; L6 – Create
SDG Mapping (SDG 1-17)	SDG 2 – Zero Hunger; SDG 9 – Industry, Innovation and Infrastructure; SDG 12 – Responsible Consumption and Production

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Agent Strategy & PEAS Analysis Document (problem formulation and strategy justification for each sub-task)
2. Software Agent Design Document (percepts, actions, internal state, agent architecture diagrams)
3. Rule-Based Expert System Implementation (knowledge base, inference engine, source code)
4. Test Cases, Demo Results & GitHub Upload

STUDENT NAMES:

KRUTHIKA SRE S - 192525360

KUPPALA TEJA SRI - 192525249

MANIYAM ANUSHKA - 192525411

A MULTI-AGENT, RULE-BASED EXPERT SYSTEM FOR CROP DISEASE DIAGNOSIS AND FARM RESOURCE ADVISORY

ABSTRACT

Agriculture requires timely decisions regarding irrigation, pest control, and crop disease management. Small-scale farmers may experience delays or inconsistencies when relying on manual advice from field agents. AgriAdvisor is an AI-based farm decision-support system designed to provide timely recommendations using software agents and a rule-based expert system.

The system consists of three intelligent agents for irrigation scheduling, pest/disease alerts, and crop-disease diagnosis. Each agent receives environmental or symptom-based percepts, processes them using an appropriate agent strategy, and interacts with a rule-based knowledge base. IF-THEN production rules are evaluated by an inference engine using forward and/or backward chaining to generate suitable recommendations. The system demonstrates how AI agents and expert-system reasoning can be integrated into a single agricultural decision-support application.

INTRODUCTION

Agricultural decision-making often depends on environmental conditions, crop symptoms, and timely intervention. Decisions such as when to irrigate, whether a pest threat exists, and which disease may affect a crop require consistent reasoning.

AgriAdvisor addresses this problem by combining intelligent software agents with a rule-based expert system. The agents observe relevant inputs, reason about the current situation, query the knowledge base, and provide appropriate recommendations.

The project focuses on three major agricultural tasks:

1. Irrigation scheduling
2. Pest/disease alerts
3. Crop-disease diagnosis

The system follows the AI concepts of PEAS formulation, intelligent-agent strategies, percepts, actions, internal state, knowledge representation, inference, and rule-based reasoning.



Fig 1: Overall concept of AgriAdvisor

PROBLEM STATEMENT

Small-scale farmers may face delayed, inconsistent, or limited access to agricultural advice. Manual decision-making can make it difficult to respond quickly to changing soil, environmental, pest, and crop-health conditions.

Therefore, the problem addressed by AgriAdvisor is:

To design and implement an AI-based multi-agent decision-support system that provides irrigation recommendations, pest/disease alerts, and crop-disease diagnosis using suitable agent strategies and a rule-based expert system.

This directly follows the problem statement in the assignment brief.

OBJECTIVES

The main objectives are:

- To formulate agricultural decision problems using the PEAS framework.
- To select suitable intelligent-agent strategies for different agricultural tasks.
- To design software agents with defined percepts, actions, and internal states.
- To construct an agricultural IF-THEN knowledge base.
- To implement an inference engine using forward and/or backward chaining.
- To integrate the agents with the expert system.
- To generate irrigation, pest-control, and disease-diagnosis recommendations.
- To test the system under different agricultural conditions.

SYSTEM OVERVIEW

AgriAdvisor follows a multi-agent architecture in which each specialized agent handles a particular agricultural task.

Main components

1. Irrigation Agent

Analyzes soil/environmental conditions and determines whether irrigation is required.

2. Pest Alert Agent

Analyses environmental conditions and observed pest-related indicators to identify possible pest/disease risks.

3. Crop Disease Diagnosis Agent

Uses crop symptoms to identify possible diseases and recommend suitable actions.

4. Rule-Based Expert System

Stores agricultural knowledge in IF-THEN production rules.

5. Inference Engine

Matches available facts against rules and derives conclusions.

6. User Interface

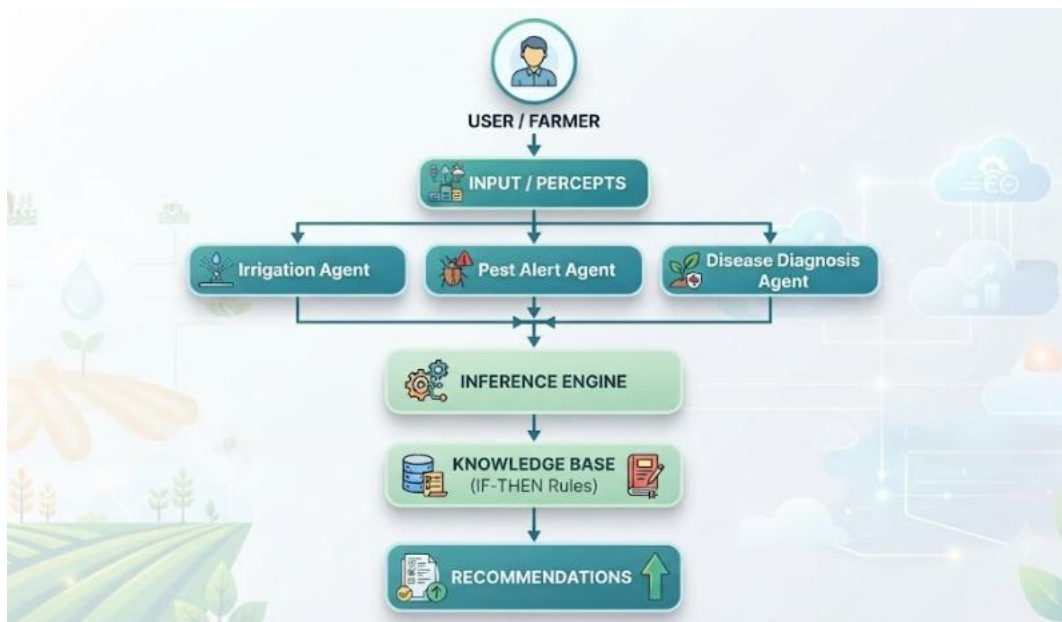


Fig 2: Overall System Architecture

PEAS ANALYSIS

The assignment specifically expects PEAS analysis and justification of the appropriate strategy for each sub-task.

6.1 Irrigation Agent

PEAS Component	Description
Performance Measure	Correct irrigation decision, reduced water wastage, healthy crop conditions
Environment	Farm, soil, crop and environmental conditions
Actuators	Irrigation recommendation / irrigation control
Sensors	Soil moisture, temperature, humidity and related readings

Agent Strategy: Model-Based Reflex Agent

A model-based reflex agent is suitable because irrigation decisions depend not only on the current sensor reading but also on the interpreted state of the agricultural environment. The agent maintains an internal representation of the farm condition and applies appropriate rules.

6.2 Pest Alert Agent

PEAS Component	Description
Performance Measure	Accurate and timely pest/disease warning
Environment	Crop field and changing environmental conditions
Actuators	Pest alert and recommended preventive action
Sensors	Environmental readings and pest-related observations

Agent Strategy: Model-Based Reflex Agent

The environment is dynamic because pest risks can change with environmental conditions. A model-based reflex approach allows the agent to maintain the relevant state and apply rules to determine whether an alert should be generated.

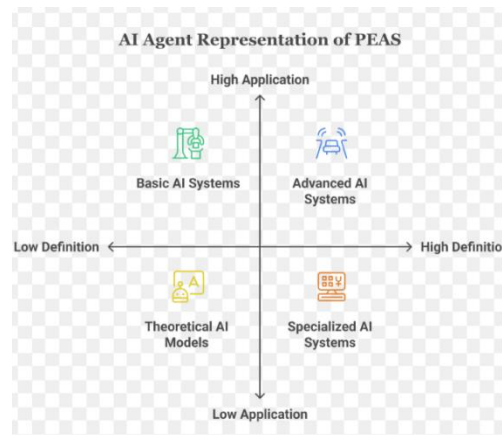


Fig 3: PEAS diagram for Pest Alert Agent

6.3 Crop Disease Diagnosis Agent

<i>PEAS Component</i>	Description
<i>Performance Measure</i>	Correct disease identification and appropriate recommendation
<i>Environment</i>	Crop and its observable symptoms
<i>Actuators</i>	Disease diagnosis and treatment/action recommendation
<i>Sensors</i>	User-provided crop symptoms and observations

Agent Strategy: Goal-Based Agent

The diagnosis agent works toward the goal of identifying the most likely crop disease from the observed symptoms. It evaluates the available facts against the knowledge base until an appropriate diagnosis and recommendation are obtained.

AGENT ENVIRONMENT ANALYSIS

Agent	Observable	Deterministic/Stochastic	Static/Dynamic
Irrigation	Partially observable	Stochastic	Dynamic
Pest Alert	Partially observable	Stochastic	Dynamic
Disease Diagnosis	Partially observable	Stochastic	Dynamic

Agricultural environments are generally dynamic because soil and environmental conditions change over time. The system therefore requires agents that can interpret changing percepts and make decisions based on the current state.

SOFTWARE AGENT DESIGN

Each agent consists of three primary elements:

Percepts → Internal Processing → Actions

Irrigation Agent

Percepts: soil moisture, temperature, humidity

Internal State: current soil/environmental condition

Action: irrigation recommendation

Pest Alert Agent

Percepts: environmental readings, pest indicators

Internal State: current pest-risk condition

Action: pest/disease alert and preventive recommendation

Disease Diagnosis Agent

Percepts: crop symptoms

Internal State: observed symptom set and possible disease states

Action: diagnosis and recommended treatment/action



Fig 4: Generic Intelligent Agent Architecture

RULE-BASED EXPERT SYSTEM

The expert system represents agricultural knowledge using IF-THEN production rules.

The knowledge base contains rules related to:

- Crop symptoms
- Environmental conditions
- Irrigation requirements
- Pest/disease conditions
- Disease diagnosis
- Recommended actions

This directly addresses the requirement for a structured IF-THEN knowledge base.

Example Rules

Irrigation Rule

IF soil_moisture = LOW

THEN irrigation = REQUIRED

Pest Alert Rule

IF humidity = HIGH

AND pest_indicator = PRESENT

THEN pest_alert = HIGH

Disease Diagnosis Rule

IF leaf_color = YELLOW

AND leaf_spots = PRESENT

THEN disease = POSSIBLE_LEAF_DISEASE

Action Recommendation Rule

IF disease = POSSIBLE_LEAF_DISEASE

THEN action = APPLY_RECOMMENDED_TREATMENT

.

```
# Irrigation
{
  "conditions": {"soil_moisture": "low"},
  "conclusion": ("irrigation", "required"),
  "message": "Irrigation is required."
},
{
  "conditions": {"soil_moisture": "adequate"},
  "conclusion": ("irrigation", "not_required"),
  "message": "Irrigation is not required."
},

# Pest Alert
{
  "conditions": {
    "humidity": "high",
    "pest_indicator": "present"
  },
  "conclusion": ("pest_alert", "high"),
  "message": "High pest risk detected."
},
{
  "conditions": {
    "humidity": "normal",
    "pest_indicator": "absent"
  },
  "conclusion": ("pest_alert", "low"),
  "message": "No major pest alert detected."
},

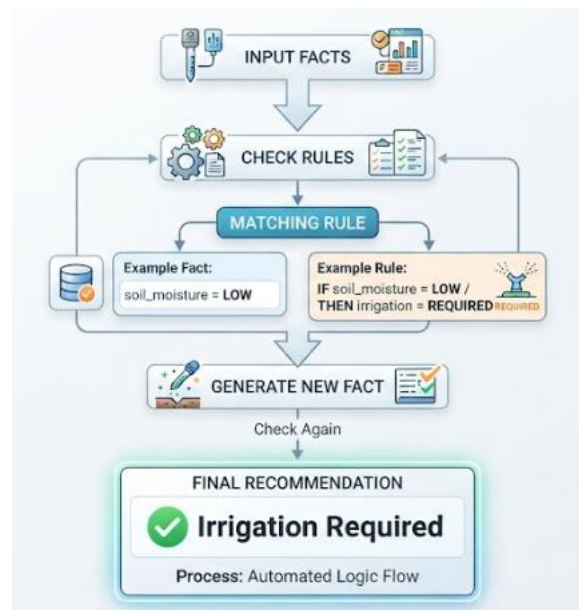
# Disease Diagnosis
{
  "conditions": {
    "leaf_color": "yellow",
    "leaf_spots": "present"
  },
  "conclusion": ("disease", "possible_leaf_disease"),
  "message": "Possible leaf disease detected."
},
{
  "conditions": {
    "leaf_color": "green",
    "leaf_spots": "absent"
  },
  "conclusion": ("disease", "healthy"),
  "message": "No major disease symptoms detected."
}
}
```

INFERENCE ENGINE

The inference engine is responsible for applying the knowledge stored in the rule base to the facts supplied by the agents.

Forward Chaining

Forward chaining starts with known facts and repeatedly applies matching rules to derive new facts.



The rubric expects the inference engine to correctly fire applicable rules, resolve conflicts where necessary, and produce accurate conclusions.

AGENT-EXPERT SYSTEM INTEGRATION

The three agents interact with the expert system to make decisions.



For example, the irrigation agent receives a low soil-moisture value, converts it into a fact, sends the fact to the inference engine, and receives an irrigation recommendation.

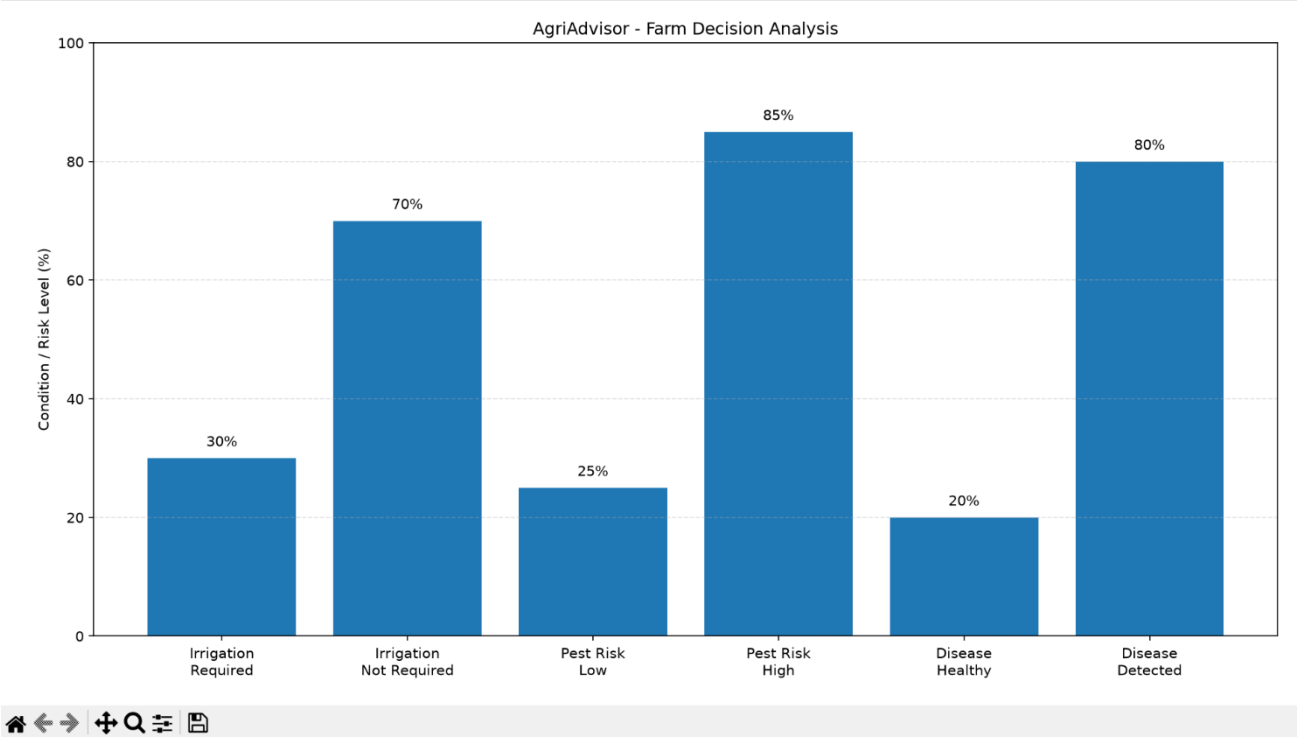
The integrated system therefore follows:

Sensing → Reasoning → Decision → Action

This is important because the rubric specifically evaluates whether agents correctly invoke the expert system and act on its recommendations end-to-end.

SYSTEM IMPLEMENTATION

- The system can be implemented as a software application consisting of:
- Agent modules
- Knowledge-base module
- Inference-engine module
- User-interface module
- Testing module
- Processing sequence
- User/environment provides input.
- Relevant agent receives the percept.
- Agent processes the input.
- Facts are generated.
- Facts are passed to the expert system.
- Inference engine evaluates applicable rules.
- A conclusion is generated.
- Agent performs the corresponding action.



TESTING AND TEST CASES

Test Case	Input Condition	Expected Output
TC01	Low soil moisture	Irrigation required
TC02	Adequate soil moisture	Irrigation not required
TC03	High pest-risk condition	Pest alert
TC04	Normal pest condition	No major pest alert
TC05	Disease-related symptoms	Possible disease diagnosis
TC06	Multiple symptoms	Corresponding disease recommendation
TC07	Normal crop condition	No disease alert

```

=====
                        AGRIADVISOR AI SYSTEM
=====

TEST CASE 1: Irrigation
Input : Soil moisture = LOW
Output: Irrigation is required.

TEST CASE 2: Irrigation
Input : Soil moisture = ADEQUATE
Output: Irrigation is not required.

TEST CASE 3: Pest Alert
Input : Humidity = HIGH, Pest indicator = PRESENT
Output: High pest risk detected. Pest control action is recommended.

TEST CASE 4: Pest Alert
Input : Humidity = NORMAL, Pest indicator = ABSENT
Output: No major pest alert detected.

TEST CASE 5: Disease Diagnosis
Input : Leaf color = YELLOW, Leaf spots = PRESENT
Output: Possible leaf disease detected.

TEST CASE 6: Disease Diagnosis
Input : Leaf color = GREEN, Leaf spots = ABSENT
Output: No major disease symptoms detected.

=====

```

RESULTS AND DISCUSSION

The developed system demonstrates the integration of intelligent agents with a rule-based expert system for agricultural decision support.

The irrigation agent identifies irrigation requirements based on environmental conditions. The pest alert agent identifies potentially risky conditions and generates alerts. The disease diagnosis agent uses observed symptoms and the knowledge base to derive a possible diagnosis and corresponding recommendation.

The inference engine provides the reasoning mechanism that connects user/environmental inputs to the final recommendations.

ADVANTAGES

- Provides consistent rule-based recommendations.
- Reduces dependence on delayed manual advice.
- Separates agricultural tasks into specialized agents.
- Supports explainable IF-THEN reasoning.
- Can process different environmental and symptom inputs.
- Provides faster decision support.

- Knowledge can be expanded by adding new rules.

LIMITATIONS

- Recommendations depend on the quality and completeness of the knowledge base.
- Rule-based reasoning cannot automatically learn new agricultural patterns unless learning mechanisms are added.
- Environmental inputs may not always represent the complete field condition.
- Disease diagnosis based only on predefined symptoms may not handle every real-world disease case.

FUTURE ENHANCEMENTS

Future versions can incorporate:

- Real-time IoT sensors.
- Image-based crop disease detection.
- Machine-learning-based prediction.
- Weather API integration.
- Mobile application support.
- Larger agricultural knowledge bases.
- Historical data analysis.

SDG MAPPING

The assignment maps the project to the following Sustainable Development Goals:

SDG 2: Zero Hunger

Supports better agricultural decision-making and crop management.

SDG 9: Industry, Innovation and Infrastructure

Demonstrates the application of AI technology to agricultural decision support.

SDG 12: Responsible Consumption and Production

Supports efficient use of agricultural resources, particularly irrigation resources.

REFLECTION AND LEARNING OUTCOMES

The project provided practical understanding of selecting suitable agent strategies according to the characteristics of an environment. Designing separate agents for irrigation, pest alerts, and disease diagnosis demonstrated how AI agents can divide a larger problem into specialized tasks.

The implementation of the rule-based expert system provided experience in knowledge representation, IF-THEN production rules, and inference mechanisms. Integrating the agents with the inference engine demonstrated how sensing, reasoning, and action can be combined into an end-to-end intelligent system.

The rubric specifically expects reflection on design decisions, challenges faced, and learning gained, rather than a generic conclusion.

CONCLUSION

AgriAdvisor demonstrates an AI-based approach to agricultural decision support by combining specialized software agents with a rule-based expert system. The system addresses irrigation scheduling, pest/disease alerts, and crop-disease diagnosis through PEAS-based agent formulation, appropriate agent strategies, IF-THEN knowledge representation, and inference-based reasoning.

The project shows how AI concepts can be applied to provide consistent and explainable agricultural recommendations while establishing a foundation for future integration with sensors, machine learning, and real-time agricultural data.

REFERENCES:

- Arumuga Arun, R., & Umamaheswari, S. (2023). Effective multi-crop disease detection using pruned complete concatenated deep learning model. *Expert Systems with Applications*, 213, 118905. <https://doi.org/10.1016/j.eswa.2022.118905>
- Routis, G., Michailidis, M., & Roussaki, I. (2024). Plant disease identification using machine learning algorithms on single-board computers in IoT environments. *Electronics*, 13(6), 1010. <https://doi.org/10.3390/electronics13061010>
- García-Vera, Y. E., Polochè-Arango, A., Mendivelso-Fajardo, C. A., & Gutiérrez-Bernal, F. J. (2024). Hyperspectral image analysis and machine learning techniques for crop disease detection and identification: A review. *Sustainability*, 16(14), 6064.
- Zhao, X., Chen, B., Ji, M., Wang, X., Yan, Y., Zhang, J., Liu, S., Ye, M., & Lv, C. (2024).

Implementation of large language models and agricultural knowledge graphs for efficient plant disease detection. *Agriculture*, 14(8), 1359.

- Gao, R., Dong, Z., Wang, Y., Cui, Z., Ye, M., Dong, B., Lu, Y., Wang, X., Song, Y., & Yan, S. (2024). Intelligent cotton pest and disease detection: Edge computing solutions with transformer technology and knowledge graphs. *Agriculture*, 14(2), 247.
- Cui, K., Huang, J., Dai, G., Fan, J., & Dewi, C. (2024). Diagnosis of custard apple disease based on adaptive information entropy data augmentation and multiscale region aggregation interactive visual transformers. *Agronomy*, 14(11), 2605. <https://doi.org/10.3390/agronomy14112605>
- Technology from traditional knowledge: Vrikshayurveda-based expert system for diagnosis and management of plant diseases. (2024). *Journal of Ayurveda and Integrative Medicine*, 15(1), 100853. <https://doi.org/10.1016/j.jaim.2023.100853>
- Trends in machine and deep learning techniques for plant disease identification: A systematic review. (2024). *Agriculture*, 14(12), 2188. <https://doi.org/10.3390/agriculture14122188>
- Food and Agriculture Organization of the United Nations. (2024, November 18). Science and Innovation Forum 2024: Digital Agriculture Changemakers in Action.
- Food and Agriculture Organization of the United Nations. (2025). Digital agriculture and AI innovation roadmap. <https://doi.org/10.4060/cd5956en>

APPENDIX A – COMPLETE SOURCE CODE

```
import matplotlib.pyplot as plt
RULES = [
    # Irrigation
    {
        "conditions": {"soil_moisture": "low"},
        "conclusion": ("irrigation", "required"),
        "message": "Irrigation is required."
    },
    {
        "conditions": {"soil_moisture": "adequate"},
        "conclusion": ("irrigation", "not_required"),
        "message": "Irrigation is not required."
    },
    # Pest Alert
    {
        "conditions": {
            "humidity": "high",
            "pest_indicator": "present"
```



```

    },
    "conclusion": ("pest_alert", "high"),
    "message": "High pest risk detected."
},
{
    "conditions": {
        "humidity": "normal",
        "pest_indicator": "absent"
    },
    "conclusion": ("pest_alert", "low"),
    "message": "No major pest alert detected."
},
# Disease Diagnosis
{
    "conditions": {
        "leaf_color": "yellow",
        "leaf_spots": "present"
    },
    "conclusion": ("disease", "possible_leaf_disease"),
    "message": "Possible leaf disease detected."
},
{
    "conditions": {
        "leaf_color": "green",
        "leaf_spots": "absent"
    },
    "conclusion": ("disease", "healthy"),
    "message": "No major disease symptoms detected."
}
]

```

```

class InferenceEngine:
    def forward_chaining(self, facts):
        conclusions = []
        for rule in RULES:
            matched = True
            for key, value in rule["conditions"].items():
                if facts.get(key) != value:
                    matched = False
                    break
            if matched:
                key, value = rule["conclusion"]
                conclusions.append({
                    "type": key,
                    "value": value,
                    "message": rule["message"]
                })
        return conclusions

class IrrigationAgent:
    def __init__(self, engine):

```

```

self.engine = engine
self.internal_state = {}

def perceive(self, soil_moisture):
    self.internal_state["soil_moisture"] = soil_moisture
def act(self)
    results = self.engine.forward_chaining(
        self.internal_state
    )
    for result in results:
        if result["type"] == "irrigation":
            return result["message"]
    return "No irrigation recommendation."
class PestAlertAgent:
    def __init__(self, engine):
        self.engine = engine
        self.internal_state = {}
    def perceive(self, humidity, pest_indicator):
        self.internal_state["humidity"] = humidity
        self.internal_state["pest_indicator"] = pest_indicator
    def act(self):
        results = self.engine.forward_chaining(
            self.internal_state
        )
        for result in results:
            if result["type"] == "pest_alert":
                return result["message"]
        return "No pest recommendation."
class DiseaseDiagnosisAgent:
    def __init__(self, engine):
        self.engine = engine
        self.internal_state = {}
    def perceive(self, leaf_color, leaf_spots):
        self.internal_state["leaf_color"] = leaf_color
        self.internal_state["leaf_spots"] = leaf_spot
    def act(self):
        results = self.engine.forward_chaining(
            self.internal_state
        )
        for result in results:
            if result["type"] == "disease":
                return result["message"]
        return "Disease could not be determined."
class AgriAdvisor:
    def __init__(self)
        engine = InferenceEngine()
        self.irrigation_agent = IrrigationAgent(engine)
        self.pest_agent = PestAlertAgent(engine)
        self.disease_agent = DiseaseDiagnosisAgent(engine)

```

```

def irrigation_check(self, moisture):

    self.irrigation_agent.perceive(moisture)
    return self.irrigation_agent.act()
def pest_check(self, humidity, pest):
    self.pest_agent.perceive(
        humidity,
        pest
    )
    return self.pest_agent.act()
def disease_check(self, color, spots):
    self.disease_agent.perceive(
        color,
        spots
    )
    return self.disease_agent.act()
system = AgriAdvisor()
# Irrigation
irrigation_cases = [
    ("Low", 30),
    ("Adequate", 70)
]
# Pest risk
pest_cases = [
    ("Low Risk", 25),
    ("High Risk", 85)
]
# Disease
disease_cases = [
    ("Healthy", 20),
    ("Possible Disease", 80)
]
print("=" * 55)
print("          AGRIADVISOR AI SYSTEM")
print("=" * 55)
print("\nIRRIGATION AGENT")
print("-----")
print(
    "Soil Moisture: LOW"
)
print(
    "Output:",
    system.irrigation_check("low")
)
print(
    "\nSoil Moisture: ADEQUATE"
)
print(
    "Output:",

```

```

    system.irrigation_check("adequate")

print("\nPEST ALERT AGENT")
print("-----")
print(
    "Humidity: HIGH | Pest Indicator: PRESENT"
)
print(
    "Output:",
    system.pest_check("high", "present")
)
print(
    "\nHumidity: NORMAL | Pest Indicator: ABSENT"
)
print(
    "Output:",
    system.pest_check("normal", "absent")
)
print("\nDISEASE DIAGNOSIS AGENT")
print("-----")
print(
    "Yellow Leaves | Leaf Spots Present"
)
print(
    "Output:",
    system.disease_check("yellow", "present")
)
print(
    "\nGreen Leaves | No Leaf Spots"
)
print(
    "Output:",
    system.disease_check("green", "absent")
)
fig, ax = plt.subplots(figsize=(10, 6))
categories = [
    "Irrigation\nRequired",
    "Irrigation\nNot Required",
    "Pest Risk\nLow",
    "Pest Risk\nHigh",
    "Disease\nHealthy",
    "Disease\nDetected"
]
values = [30, 70, 25, 85, 20, 80]
bars = ax.bar(categories, values)
ax.set_title(
    "AgriAdvisor - Farm Decision Analysis"
)
ax.set_ylabel(

```

```
    "Condition / Risk Level (%)"
)
ax.set_ylim(0, 100)
ax.grid(axis="y", linestyle="--", alpha=0.4)
for bar, value in zip(bars, values):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        value + 2,
        f"{value}%",
        ha="center"
    )
plt.tight_layout()
plt.show()
```

Rubric

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Formulation & Agent Strategy Selection (CO3)	15	Correctly analyses the problem environment (PEAS: Performance, Environment, Actuators, Sensors); justifies the most apt agent strategy (reflex, model-based, goal-based, or utility-based) with clear, well-reasoned rationale for each sub-task.	PEAS description and agent strategy mostly correct with minor gaps in justification.	PEAS description present but agent strategy choice is only partially justified.	PEAS/agent strategy missing, incorrect, or not matched to the problem environment.
Software Agent Design & Architecture (CO4)	20	Software agents are clearly designed with well-defined percepts, actions, and internal state/goals; agent architecture (e.g., model-based reflex, goal-based, learning) is appropriately structured and clearly diagrammed for each module (irrigation, pest alert, diagnosis).	Agents are designed with minor gaps in percept/action definition or architecture clarity.	Basic agent design present but architecture is shallow or loosely connected to the problem.	No meaningful agent design, or agents do not correspond to the stated tasks.
Knowledge Base Construction (Rule-Based Expert System) (CO5)	15	Knowledge base contains a comprehensive, well-structured set of IF-THEN production rules covering crop diseases, symptoms, and recommended treatments, correctly encoded and free of redundancy or conflict.	Knowledge base is reasonably complete with minor rule redundancy or gaps.	Knowledge base covers only limited cases; some rules are poorly structured.	Knowledge base is missing, minimal, or rules are incorrect/inconsistent.
Inference Engine Implementation (CO5)	20	Inference engine correctly implements forward and/or backward chaining; correctly fires applicable rules, resolves conflicts, and arrives at accurate diagnoses/recommendations for a range of test inputs.	Inference engine works correctly for most test cases with minor logical errors.	Inference engine partially implemented; works only for simple/limited cases.	Inference engine missing, non-functional, or produces incorrect conclusions.
Agent – Expert System Integration (CO3/CO4/CO5)	15	Agents correctly invoke the rule-based expert system as needed and act on its recommendations; the overall system behaves coherently end-to-end across sensing, reasoning, and acting.	Integration mostly works with minor inconsistencies between agent actions and expert-system output.	Agents and expert system operate but integration is loose or requires manual intervention.	Agents and expert system are disconnected or do not function together.
Testing, Demonstration & Result Analysis	10	Comprehensive test cases cover diverse scenarios (multiple crops/diseases/environmental conditions); results are clearly demonstrated and analysed with correct/expected outputs.	Reasonable set of test cases with mostly correct results; minor gaps in analysis.	Limited test cases; results demonstrated but analysis is shallow.	Little to no testing or demonstration; results not verified.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Reflection: Design Decisions & Learning Outcomes	5	Thoughtful justification of agent-strategy and rule-base design choices; honest, specific account of challenges faced and learning gained.	General justification of design choices; challenges and learning mentioned but lack depth.	Reflection present but generic; limited justification of design or vague learning outcomes.	No genuine reflection submitted, or content is generic with no real design connection.